University:

Course: Deep Learning

Laboratory Work 2

Topic: Perceptron – Binary Classification

Student: Rustam Khalimov

Student ID: 23B031473

Instructor: Yerlan Kuzbakov

Date: 05.02.2026

Variant: 6

Variant 6

Problem 6.1

In this problem we have weights:

$w_1 = -0.4$

$w_2 = 0.6$

$w_3 = 0.3$

and bias which equal to 0.0 ($b = 0.0$)

There we should find output of vector (for example) $x = (0.5 , 0.3 , 0.8)$

Desired output is $d = 1$ and speed of training $eta = 0.9$

Firstly we calculate the weighted sum by using this formula:

$z = w_1 * x_1 + w_2 * x_2 + w_3 * x_3 + b$

$z = -0.4 * 0.5 + 0.6 * 0.3 + 0.3 * 0.8 + 0.0 = -0.2 + 0.18 + 0.24 = 0.22$

Then apply the activation function:

Our $z = 0.22 > 0$ therefore $y = 1$

After that we compare it with desire output:

$d = 1$ and $y = 1$

$d - y = 1 - 1 = 0$

No weight update required.

Updating weights:

Our error = 0 so:

$w_i(new) = w_i(old)$ it means weights doesn`t change.

Problem 6.2

```
import numpy as np
import pandas as pd
import sklearn as sk
import matplotlib # I import kust mathplotlib cuz mathplotlib.pyplot dont have "__version__"
import matplotlib.pyplot as plt
import warnings
warnings.filterwarnings('ignore')
from sklearn.linear_model import LogisticRegression
from sklearn.metrics import confusion_matrix ,  classification_report , accuracy_score
from sklearn.linear_model import Perceptron
```

```
print("NumPy:", np.__version__)
print("Pandas:", pd.__version__)
print("Scikit-learn:", sk.__version__)
print("Matplotlib:", matplotlib.__version__)
```

```
NumPy: 2.4.1
Pandas: 3.0.0
Scikit-learn: 1.8.0
Matplotlib: 3.10.8
```

Figure 1. Here librias which I use in this lab and their versions

```
# Problem 6.2
```

```
X_train = np.array([
    [0.9 , 0.8],
    [0.2 , 0.1],
    [0.7 , 0.9],
    [0.3 , 0.2],
    [0.8 , 0.7],
    [0.1 , 0.3]
])
```

```
y_train = np.array([1 , 0 , 1 , 0 , 1 ,0])
```

Figure 2. We can see train data for our problem

A) Train using eta0=1.0, max_iter=100, random_state=42.

```
model = Perceptron(
    eta0 = 1.0, #learning rate
    max_iter = 100, # maximum epochs
    random_state = 42, #for reproducinility
    verbose = 1 #print progress
)
```

```
model.fit(X_train , y_train)
```

```
-- Epoch 1
Norm: 0.85, NNZs: 2, Bias: 0.000000, T: 6, Avg. loss: 0.223333, Objective: 0.223333
Total training time: 0.00 seconds.
-- Epoch 2
Norm: 1.70, NNZs: 2, Bias: 0.000000, T: 12, Avg. loss: 0.118333, Objective: 0.118333
Total training time: 0.00 seconds.
-- Epoch 3
Norm: 1.49, NNZs: 2, Bias: -1.000000, T: 18, Avg. loss: 0.060000, Objective: 0.060000
Total training time: 0.00 seconds.
-- Epoch 4
Norm: 1.49, NNZs: 2, Bias: -1.000000, T: 24, Avg. loss: 0.000000, Objective: 0.000000
Total training time: 0.00 seconds.
-- Epoch 5
Norm: 1.49, NNZs: 2, Bias: -1.000000, T: 30, Avg. loss: 0.000000, Objective: 0.000000
Total training time: 0.00 seconds.
-- Epoch 6
Norm: 1.49, NNZs: 2, Bias: -1.000000, T: 36, Avg. loss: 0.000000, Objective: 0.000000
Total training time: 0.00 seconds.
-- Epoch 7
Norm: 1.49, NNZs: 2, Bias: -1.000000, T: 42, Avg. loss: 0.000000, Objective: 0.000000
Total training time: 0.00 seconds.
-- Epoch 8
Norm: 1.49, NNZs: 2, Bias: -1.000000, T: 48, Avg. loss: 0.000000, Objective: 0.000000
Total training time: 0.00 seconds.
-- Epoch 9
Norm: 1.49, NNZs: 2, Bias: -1.000000, T: 54, Avg. loss: 0.000000, Objective: 0.000000
Total training time: 0.00 seconds.
Convergence after 9 epochs took 0.00 seconds
```

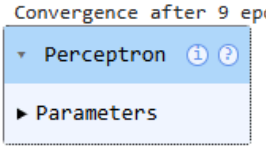

Figure 3. Perceptron model and its training process.

Interpretation:

Below you can see the "Epoch" values, which indicate the stages of training the model. And in the end, it is clear that after the 9th epoch or stage, the model stopped making mistakes, which indicates its successful convergence.

B) Determine if the data is linearly separable based on convergence behavior and final accuracy.

```
#Displaying learned parameters
print("Learned weights (w_1 , w_2):" ,  model.coef_)
print("Learned bias(b):" , model.intercept_)
print("Numner of iterations:" , model.n_iter_)
```

```
Learned weights (w_1 , w_2): [[1.  1.1]]
Learned bias(b): [-1.]
Numner of iterations: 9
```

Figure 4. Displaying learned parameters

```
#Generating predictions
y_pred = model.predict(X_train)
```

Figure 5. Generating predictions

```
#Calculating accuracy
acc = accuracy_score(y_train , y_pred)
print("Training accuracy:" , acc * 100 , "%")
```

```
Training accuracy: 100.0 %
```

Figure 6. Calculating accuracy

```python
#Predictions VS Actual
print("Sample-by-sample results:")
for i in range (len(y_train)):
    if y_train[i] == y_pred[i]:
        status = "Correct"
    else:
        status = "Incorrect"
    print( "Sample = " , i + 1 , ":")
    print( "Actual = " , y_train[i])
    print( "Predicted = " , y_pred[i] , "--" , status)
```

```
Sample-by-sample results:
Sample =  1 :
Actual =  1
Predicted =  1 -- Correct
Sample =  2 :
Actual =  0
Predicted =  0 -- Correct
Sample =  3 :
Actual =  1
Predicted =  1 -- Correct
Sample =  4 :
Actual =  0
Predicted =  0 -- Correct
Sample =  5 :
Actual =  1
Predicted =  1 -- Correct
Sample =  6 :
Actual =  0
Predicted =  0 -- Correct
```

Figure 7. Predictions VS Actual

Interpretation:

Here we compare the model's predictions with the actual values. As we can see, the results are good.

```
#Generating and displaying confusion matrix
```

```
#Compute confusion matrix
con_mat = confusion_matrix(y_train , y_pred)
print("Confusion matrix")
print(con_mat)

# Plot confusion matrix
plt.figure(figsize=(5, 4))
plt.imshow(con_mat)
plt.title("Confusion Matrix")
plt.colorbar()

plt.xlabel("Predicted label")
plt.ylabel("Actual label")

for i in range(con_mat.shape[0]):
    for j in range(con_mat.shape[1]):
        plt.text(j, i, con_mat[i, j],
                 ha="center", va="center")

plt.tight_layout()
plt.show()
```

Figure 8. Generating and displaying confusion matrix

```
Confusion matrix
[[3 0]
 [0 3]]
```

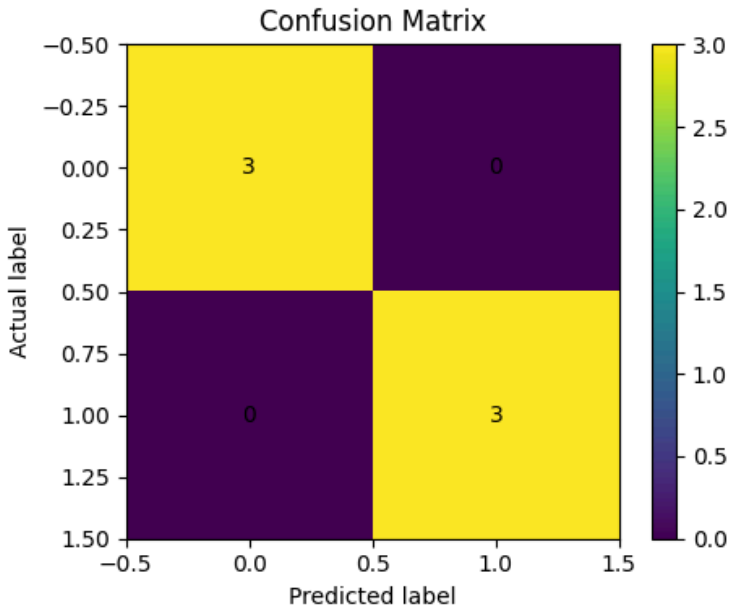

Figure 9. Plot of my models Confusion Matrix

Interpretation:

In this matrix, you can see that all values are on the main diagonal, and there are no classification errors. This means that the model correctly predicted all the objects in the training sample.

```
#Displaying classifaction report
print("Classification report")
print(classification_report(y_train , y_pred, target_names = ["Rejected" , "Approved"]))
```

```
Classification report
              precision    recall  f1-score   support

    Rejected       1.00      1.00      1.00         3
    Approved       1.00      1.00      1.00         3

    accuracy                           1.00         6
   macro avg       1.00      1.00      1.00         6
weighted avg       1.00      1.00      1.00         6
```

Figure 10. Displaying classification matrix

Interpretation:

In this screenshot, we can see the classification report. Where all indicators are equal to 1.00. This means that the model has correctly classified all the data without errors.

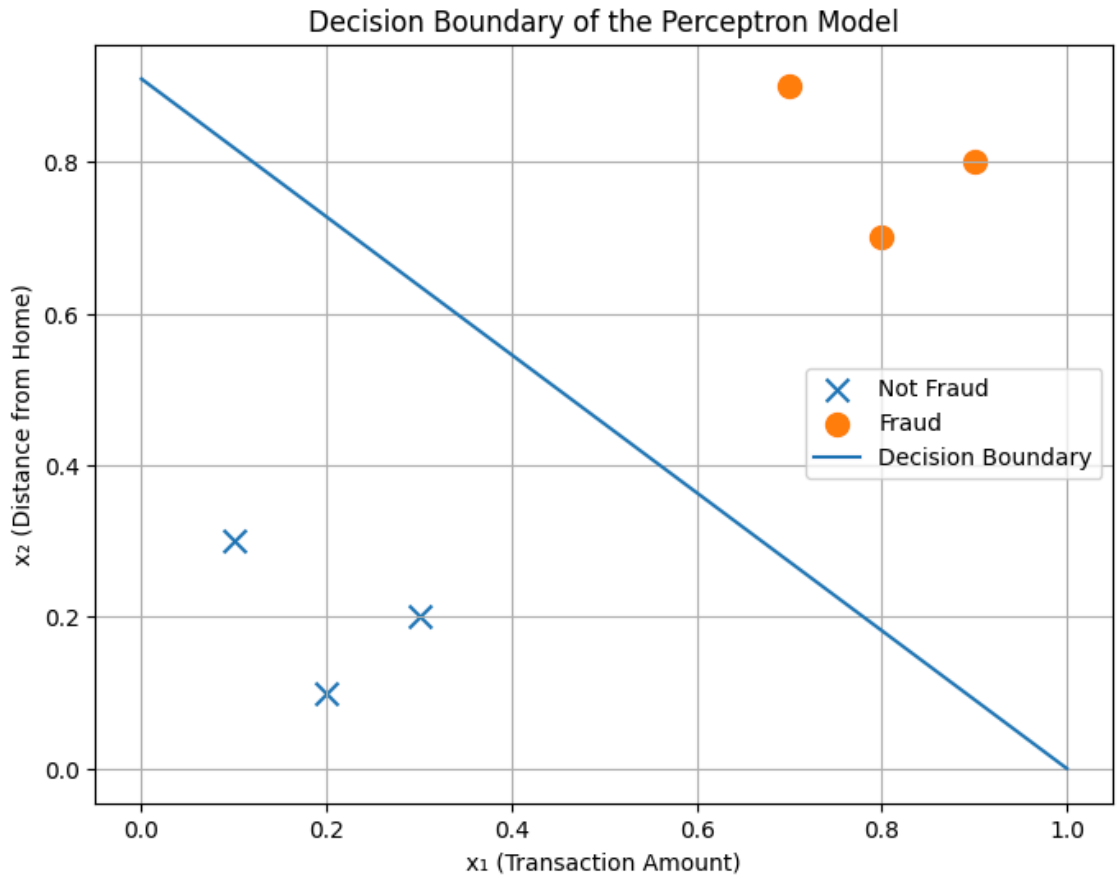

Figure 11. Decision boundary

c) Classify a transaction with x_1 = 0.6 and x_2 = 0.5.

```
#C)
```

```
X_new = np.array([[0.6, 0.5]])
prediction = model.predict(X_new)

if prediction[0] == 1:
    print("Fraud")
else:
    print("Not Fraud")
```

```
Fraud
```

Figure 12. Classifying new data

Interpretation:

In this example, the model is used to classify a new transaction with parameters $x_1 = 0.6$ and $x_2 = 0.5$. According to the prediction result, the transaction was determined to be fraudulent.

Records of results in my laboratory notebook:

Learned weight w_1: 1.0

Learned weight w_2: 1.1

Learned bias b: -1.0

Training accuracy: 100%

Number of iterations to converge: 9

Problem 6.3

We know from the conditions of the problem that:

Total transactions per month: 10,000

The share of fraud: 1%

False Positive rate (FP): 2%

False Negative rate (FN): 15%

Cost of False Positive: $25

Cost of False Negative: $1,500

And we need to find the expected monthly financial losses.

First, we determine the number of fraudulent and normal transactions. We know that the share of fraud: 1% of total transactions per month. It means the number of fraudulent transactions is equal to: 10.000 * 0.01 = 100.

Then we find normal transaction: 10.000 - 100 = 9.900

Later, we count the number of classification errors.

FN:

100 * FNR = 100 * 0.15 = 15

FP:

9.900 * FPR = 9.900 * 0.02 = 198

Then we calculate the financial losses.

Losses due to False Negatives:

15 * Cost of False Negative = 15 * 1500 $ = 22.500 $

Losses due to False Positives:

198 * Cost of False Positive = 198 * 25 $ = 4.950 $

Total monthly losses: 22.500 $ + 4.950 $ = 27.450 $

ANALYSIS QUESTIONS

1. The XOR (exclusive-or) problem consists of four points: (0,0)->0, (0,1)->1, (1,0)->1, (1,1)->0. Explain why a single-layer perceptron cannot learn this function. Include a sketch showing why no single straight line can separate the classes.

The perceptron cannot solve the XOR problem because the data is not linearly separable. For XOR, it is impossible to draw one straight line that separates classes 0 and 1.

2. What is the effect of the learning rate (eta0) on convergence speed and stability? What problems might arise if eta0 is set too high or too low? Suggest an appropriate range for eta0.

The learning rate affects how quickly and stably the model converges. If the eta0 is too large, learning can be unstable, and if it is too small, it can be too slow. Usually, the appropriate eta0 range is between 0.1 and 1.0.

3. In business applications, misclassification costs are often asymmetric (false positives and false negatives have different costs). How might you modify the perceptron training procedure or decision threshold to account for asymmetric costs?

With asymmetric errors, the decision threshold can be changed or false positive and false negative errors can be penalized differently during training.

4. Compare the scikit-learn Perceptron to LogisticRegression. What are the similarities and differences in their outputs? Which would you prefer for a business classification problem and why?

The perceptron and logistic regression use a linear model, but the perceptron provides only a binary result, while logistic regression provides a probability. For business tasks, I would choose logistic regression, as it gives more interpretable results.

5. If a perceptron fails to converge after max_iter epochs (you see a ConvergenceWarning), what does this indicate about the training data? What alternative approaches or model modifications might you consider?

If the perceptron does not converge, it means that the data is not linearly separable. In this case, you can use more complex models or add additional features.